Holonic Semantic Governance Architecture (HSGA): Version 5.0

Nov 9, 2026 · @jeffrey

1. Document control

This is version 5.0, issued 2026-11-09, of the HSGA architecture, the first to fold the lexicon layer, the graph naming standard, tenancy and the logic single source of truth into the v4.2 reference architecture.

Field	Value
Document	Holonic Semantic Governance Architecture (HSGA)
Current version	5.0, issued 2026-11-09; number proposed for a major change to the label model and asset taxonomy
Supersedes	v4.2, Reference Topology and Governance Diagram Edition
Owner	Jeffrey Wallk
Status	Issued for review, not yet reviewed by the governance council
Companion	HSGA Explorer, an interactive walkthrough of this document
Review cadence	Proposed: at each delivery gate (G0 21 Dec 2026, G1 8 Mar 2027, G2 3 May 2027, G3 24 May 2027) and on any change class below

Versioning rules

- **Number.** MAJOR.MINOR. A major change alters an invariant, the asset types or tiers, a MUST rule in the naming standard, or the service contract model. A minor change adds or extends a capability without breaking an invariant. Wording and correction fixes do not take a new number; record them in the version history as a revision note.

- **Status.** Draft, In review, Approved, Superseded. Only the council moves a version to Approved.
- **History.** Never edit a past row of the version history. Add a new row at the top.
- **Decisions.** Every open or proposed decision lives in the decision register (section 13). A version is not Approved while a decision it depends on is open.

2. Version history

Version 5.0 is the newest row; add each later version above it.

Version	Date	Class	What changed	Sources
5.0	2026-11-09	Major	Adds the lexicon layer (labels and customer terms in scoped graphs beside the core); five asset types and four tiers; the graph naming standard; tenancy, personas and access decisions; twelve semantic services; operations extensions (screens, monitoring, triage, simulation); the logic single source of truth; a complete one-page architecture diagram and five detailed breakdown diagrams (services, gateway, label resolution, logic pipeline). Core and domain terms keep preferred labels and a vetted universal set only.	Alternate Labels proposal (31 Oct 2026); Graph Naming Standard insert (1 Nov 2026); SSOT Logic Architecture

Version	Date	Class	What changed	Sources
4.2	not recorded	Minor	Reference topology and governance diagram edition: all seven holarchy layers, four governance domains, human-in-the-loop diagram set (Section 13 of v4.2).	HSGA v4.2
4.0	not recorded	Minor	Reference topology and diagram set introduced.	HSGA v4.2
3.0	not recorded	Major	Semantic Abstraction and Mediation Layer: mapping graph as a persistent lateral holon; compiled semantic governance; cross-silo entity and URI resolution; RAG grounding.	HSGA v4.2

Versions before 3.0 and the dates of 3.0 to 4.2 are not in the source documents. Fill them in if you hold them.

Changes in 5.0 that alter earlier statements

- Alternate, acronym, persona and hidden labels on core and domain terms are replaced by the vetted universal set; everything else moves to lexicons.
- Persona label graphs become scoped lexicons at the domain, enterprise or customer tier.
- The single-letter asset codes gain L (lexicon) and M (mapping).
- Tenant extensions are customer-tier vocabularies or taxonomies and are never ontologies.

3. Purpose and scope

HSGA is a governed layer of shared meaning that sits on top of existing systems, binds what exists to agreed concepts, enforces rules once, and lets AI act within limits that people can audit. It is a semantic scaffold, not a replacement platform.

In scope

- The holonic structure and seven-layer holarchy, and the four governance domains.
- The semantic design: core, domain, enterprise and customer assets, with their naming.
- The lexicon layer for alternate labels and customer terminology.
- Tenant isolation, personas, access decisions and provisioning.
- The semantic services and the operations extensions built on them.
- The single source of truth for business logic.
- Runtime governance (gateway, compiled rules, agents) and measurement.

Out of scope

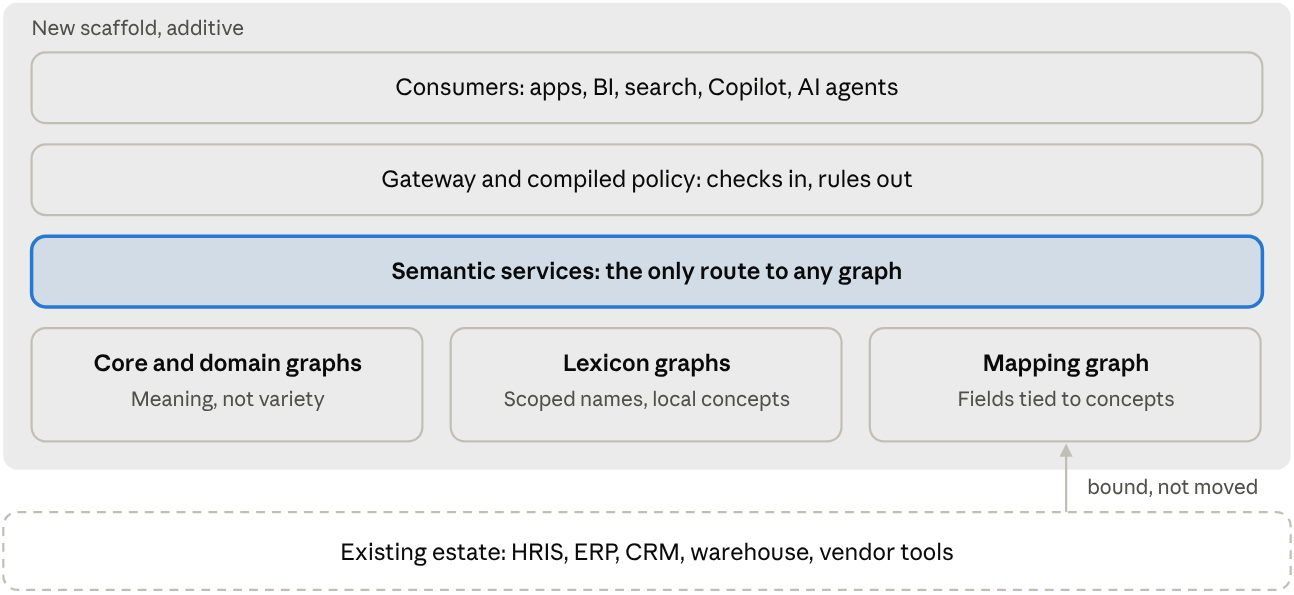
- Choice of graph store, identity provider, policy engine, telemetry platform and review-application host. The design is stack-independent; GraphDB is assumed only for sizing.
- The persona and job catalog, which Personas & Jobs owns.
- Customer data models in systems of record, which are bound, not changed.

Readers. Council members and domain stewards read sections 3 to 7 and 13. Engineers read sections 5 to 12. Operations leads read sections 8, 9 and 12.

4. Architecture at a glance

HSGA is additive: new layers of mapping, shared meaning, scoped terminology, governance and delivery sit above the legacy estate, which is bound to concepts rather than replaced.

Governed meaning sits above systems that stay in place



scaffold over legacy · 4 new layers, 1 existing estate

Requests and AI actions pass the gateway, then the services, which are the only route to any graph. The mapping graph ties system fields to concepts without moving data.

4.1 Flow

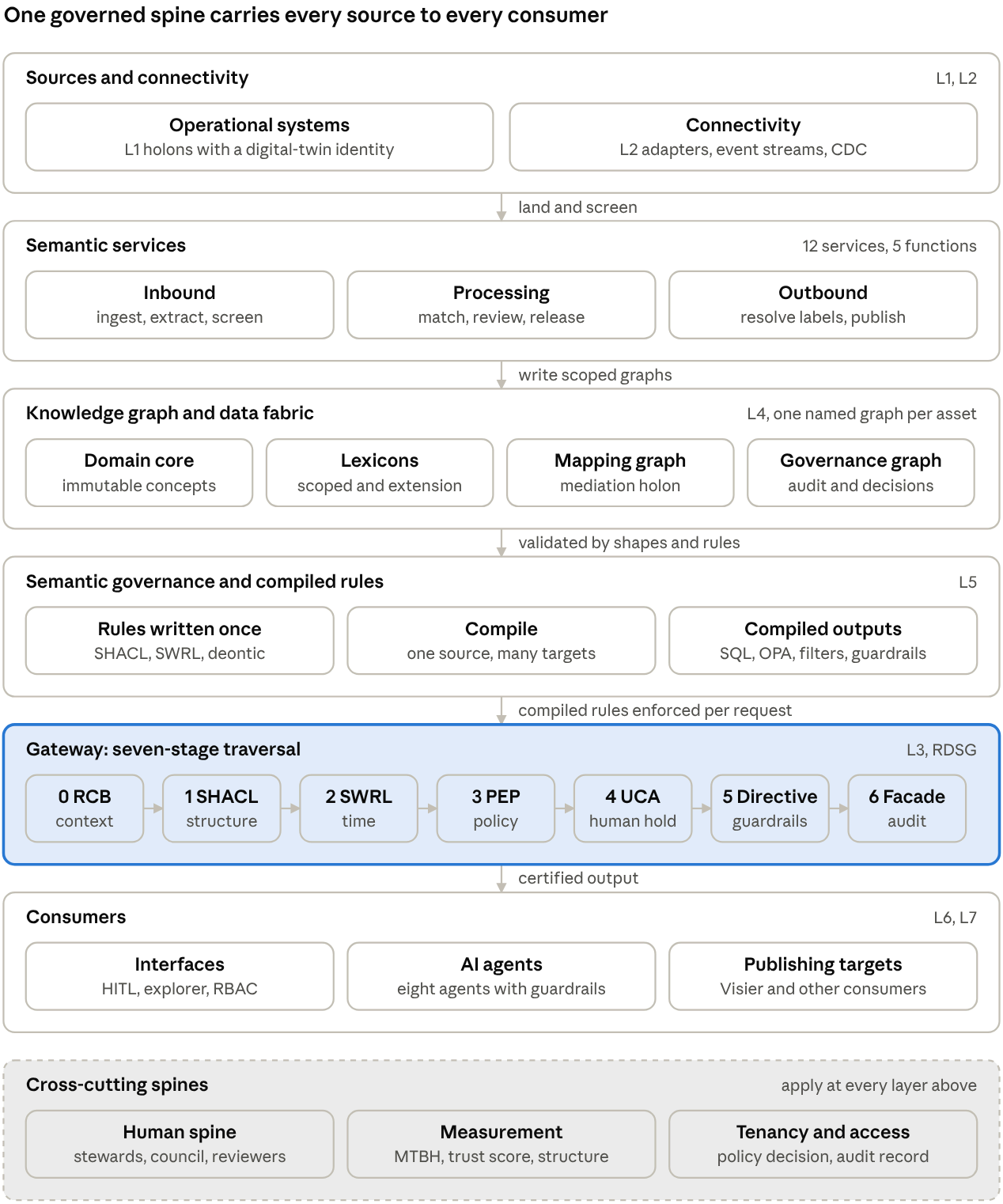
1. **Bind and map.** A mediation holon ties each system field to one agreed concept, with confidence scores and steward confirmation.
2. **Govern core and lexicons.** A small immutable core carries meaning; scoped lexicons and customer extensions carry the many names.
3. **Check at the gateway.** Every request and AI action passes seven stages.
4. **Compile and deliver.** Rules are written once and compiled to data checks, access policy, search filters, agent guardrails and rules-engine artifacts; released vocabularies publish to targets such as Visier.
5. **Human and measurement spine.** People decide what machines cannot; trust and structure are measured.

4.2 Design principles

- Add a layer; do not replace systems of record.
- Keep the core small: onboarding a customer adds no triples to it.
- Scope terminology by where it is stored, one named graph per tenant.

- Author each rule once and generate its runtime forms.
- Machines propose, people decide.
- Measure trust instead of assuming it.
- Reach every graph through a service, never directly.

4.3 Complete architecture on one page

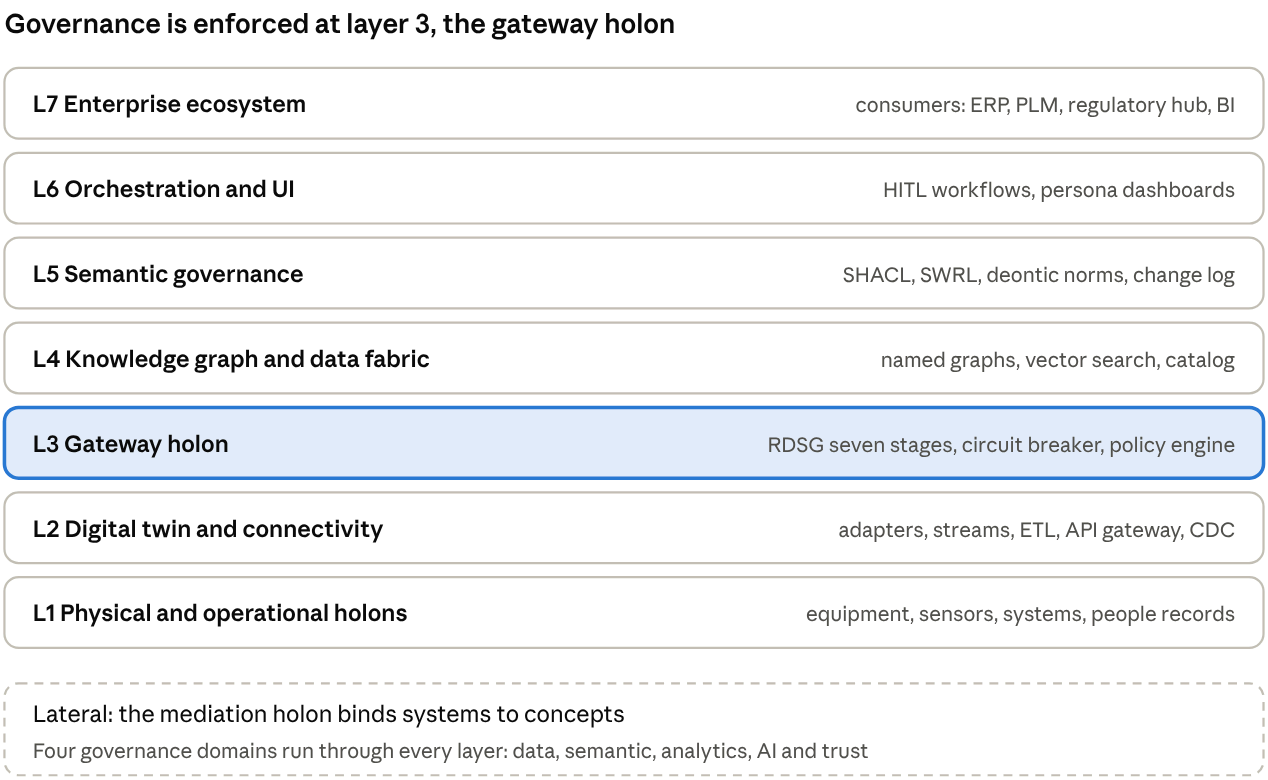


complete architecture · 6 layers in the request path, 3 cross-cutting spines

Read the diagram top to bottom. Sources feed the semantic services, which write scoped graphs into the knowledge graph. Rules live once at L5 and compile to enforcement. The gateway is the single path every request and AI action takes, and only certified output reaches consumers. The human, measurement and tenancy spines apply at every layer. Sections 5 to 12 break each band down.

5. Holonic structure

A holon is simultaneously a self-contained whole and a composable part; HSGA applies that to seven nested layers that are autonomous in their own scope and accountable to the layer above, following the Wymore system model M = (X, Y, W, S, Δ, λ, R).



seven-layer holarchy · 7 layers, 1 lateral holon

Layers 1 to 7 run from operations up to the enterprise ecosystem; the mediation holon is lateral, not an eighth layer.

5.1 Layers

Layer	What it holds	Wymore role
L7 Enterprise ecosystem	Consumers; every output traces to L4 and passes domain certification	Y output space
L6 Orchestration and UI	HITL workflows, intervention pane, SHACL viewer, ontology explorer, ontological RBAC	λ output function
L5 Semantic governance	SHACL library, SWRL temporal rules, deontic norms, ORSD-style contracts, KGCL change log	R performance specs
L4 Knowledge graph and data fabric	Triplestore with one named graph per asset (domain, lexicon, customer extension, governance), federated query, vector search, lakehouse, metric layer, catalog and lineage	W state space
L3 Gateway holon	RDSG traversal, trust circuit breaker, policy decision engine, safety specification, rule projection runtime	Δ transition function
L2 Digital twin and connectivity	Protocol adapters, twin registry, event streaming, ETL, API gateway, CDC; signals typed as RDF conformant with L5	S admissible stimuli
L1 Physical and operational holons	Equipment, sensors, operational systems; each holon has a digital-twin identity with SHACL-governed state boundaries	X input space

5.2 Four governance domains

They run through every layer: **data** (catalog, lineage, quality gates, master data, access policy), **semantic** (ontology registry, SHACL, term lifecycle, external alignment), **analytics** (model registry, one governed definition per metric, certified dashboards), and **AI and trust** (trust scoring, circuit breaker, model cards, mandatory human gates).

5.3 Mediation holon

The mapping graph is a persistent lateral holon that insulates the top-down ontology from bottom-up implementations: RAG and vector stores, property graphs, dbt and LookML models, third-party ontologies and catalogs. It holds six correspondence types, scores each with confidence, and routes low-confidence mappings to a steward before promotion. It also separates the definition of governance rules from their runtime execution (section 11).

6. Semantic design and naming standard

Every governed asset is one of five types at one of four tiers, and its name never carries a customer name, an environment, a version or a repository, so graphs can move without being renamed.

6.1 Design invariants

1. **Immutable core.** Published core terms are never edited; deprecate with `supersededBy` and replace. Evolution is additive: add a parent, never reparent.
2. **Primitives are rare and rooted.** Minted only at core or a top-level domain, with BFO lineage, kept minimal.
3. **Rooted acyclic derivation DAG.** Multi-parent inheritance is allowed and limited; report any concept with more than two parents.
4. **Three edge types, default deny.** Derivation, correspondence and lexicalization; an allowed-edge matrix by graph pair avoids a fully meshed graph.
5. **Core holds meaning; lexicons hold names.** Core and domain terms keep preferred labels, definitions, relations and a small vetted set of universal alternate labels.
6. **Three orphan checks.** Derivation path to BFO, schema binding coverage, label resolution; run per graph and per release, including each tenant graph.
7. **Five asset types, no modules.** Vocabulary, taxonomy, ontology, lexicon, mapping. A lexicon holds labels only; a mapping holds match relations between assets.
8. **Four tiers with fixed content.** Customer-tier assets are never ontologies; onboarding a customer adds no triples to a core or domain graph.
9. **Scope lives in the graph.** One named graph per tenant; references point one way, from lexicon or extension up to core, by IRI.

6.2 Types by tier

Tier	Allowed types	Holds
Core	vocabulary, taxonomy, ontology, mapping	Preferred labels, definitions, relations, a small vetted set of universal alternate labels
Domain	vocabulary, taxonomy, ontology, lexicon, mapping	The same for one domain, plus labels common to that domain but not universal; operational domains follow this tier
Enterprise	vocabulary, taxonomy, lexicon, mapping	House terms valid for every tenant
Customer	vocabulary, taxonomy, lexicon, mapping	One tenant's labels and local concepts; never an ontology

6.3 Asset names

An asset name is lowercase ASCII words joined by hyphens: `<type>-<tier>-<name>`, or `<type>-customer-<tenant-id>-<name>`. A tenant ID is `t` plus 8 lowercase hex characters, opaque, issued at provisioning and never reused. The name part is proposed at 40 characters or fewer. A customer lexicon may omit the name part, as in `lexicon-customer-t7f3a9c21`.

Example (illustrative)	What it is
`vocabulary-core-party`	A core vocabulary
`taxonomy-domain-workforce-advisory`	A domain taxonomy
`ontology-domain-payroll-operations`	An operational-domain ontology
`lexicon-enterprise-insperity`	Insperity house terms
`lexicon-customer-t7f3a9c21`	One tenant's labels
`mapping-customer-t7f3a9c21-plan-names-to-benefit-classes`	One tenant's match relations to domain concepts

6.4 Graph classes and IRIs

Every graph IRI derives from the asset URI, which is version-free and ends in the asset name. The path segments `draft`, `sim`, `changes` and `catalog` are reserved; a version segment always starts with a digit, so they cannot collide.

Graph class	IRI	Lifetime
Current release	Asset URI	Replaced by each release
Version	Asset URI + `/` + SemVer	Permanent
Draft	Asset URI + `/draft/` + change ID	Deleted after release or rejection
Simulation	Asset URI + `/sim/` + run ID	Deleted when the run record is closed; never writes to a current or version graph
Change records	Asset URI + `/changes`	Permanent, append-only
Catalog	Base URL + `/catalog`	Permanent

Repository IDs are environment-neutral: `skl-core-domain`, `skl-lexicons`, `skl-extensions`, and `skl-tenant-` plus the tenant ID for a dedicated repository. The environment belongs to the server address. Moving an asset between repositories changes only its catalog record.

6.5 Naming rules

#	Rule	Strength
1	Lowercase ASCII letters, digits and hyphens; standard or customer form	MUST
2	Type is one of the five; the word "module" is not used	MUST
3	Core assets hold no customer-specific or enterprise-specific content	MUST NOT

#	Rule	Strength
4	No customer name, environment, version, date, status or repository name in a name	MUST NOT
5	A published name never changes; a rename is a new asset with the old one retired and its replacement recorded	MUST NOT
6	Current-release IRI equals the asset URI; archived releases use version IRIs	MUST
7	Tenant ID is opaque, issued once, never reused	MUST
8	Services derive reachable graphs from caller tenant and persona; no graph IRI is accepted from a caller or screen	MUST
9	A lexicon declares the assets it labels; labels point to concepts by IRI and never copy them	MUST
10	A domain asset declares its parent core assets in metadata; parentage is not in the name	SHOULD
11	Operational domains follow the domain tier; the first word of the name is the domain	MUST
12	A draft graph is deleted when its change is released or rejected	MUST
13	Prefixes are lowercase and unique across repositories; a customer-tier prefix begins with the tenant ID	MUST / SHOULD

The Metadata Standard changes in five places: term labels, asset types, new fields (tier, tenant ID, labelled assets, mapping source and target), the catalog record, and the glossary (add lexicon, mapping, tier, tenant ID, draft graph; codes L and M).

7. Lexicon layer and customer terminology

Alternate labels and customer terms are held in scoped lexicons and customer extension vocabularies beside the core, so onboarding a customer adds no triples to the core, a label change never forces a core release, and one customer's terms never reach another.

7.1 Why labels do not live on the core

Putting every synonym on the core concept fails five ways: the core grows with every customer (growth); every label change needs a core release (churn); one customer's terms become visible to all (leakage); a flat list cannot say that the same word means different things to different customers (ambiguity); and a bare label records no source, approver or dates (lost evidence). Some terms are not synonyms at all and need a home that does not widen the core.

Option	Core size and speed	Tenant confidentiality	New customer meaning	Verdict
A. Labels on core concepts	Weak: grows with every customer	Weak: all tenants see all labels	Weak	Reject for customer labels; keep for a small vetted promoted set
B. Scoped lexicons	Strong: core unchanged	Strong: scope is the graph	Weak: names existing concepts only	Adopt for label-only terms
C. Customer extension vocabularies	Strong: core unchanged	Strong	Strong	Adopt for new local concepts
D. A copy of the core per customer	Weak: copies multiply	Strong	Partial	Reject: copies drift

B and C together, chosen by the triage rule, are the decision. B costs a resolution service and a derived index; C costs steward review of each match. Both are paid once per platform or once per concept.

7.2 Principles

1. The core holds meaning, not vocabulary variety.
2. A label is a record, not a property.
3. Scope is enforced by where a label lives.
4. References point one way, by ID; nothing is copied into the core.

5. Machines propose, people decide.
6. Derived is not source: the resolution index is rebuilt, never edited.
7. Every change is traceable to source, evidence, reviewer, release and publish record.
8. One contract per function, an adapter per source or target.
9. No direct access to a graph; every read and write goes through a service or a provisioned screen.

7.3 The label record

A label is a resource modeled in the style of SKOS-XL, with PROV-O provenance and SHACL shapes validating each record.

Field	Holds
Label text and language	The literal form
Concept	ID of the core, domain or extension concept it names
Type	Alternate, hidden (match only, never shown), abbreviation, misspelling or deprecated
Scope and tenant	Customer, Insperity or domain; tenant ID for customer labels
Evidence	Source, document or page, span, frequency
Confidence and triage class	Machine score and proposed class
Status	Proposed, in review, approved, published, rejected or deprecated
Reviewer and time	Who decided, when and why
Effective dates and release	When it applies and the SKL release that carried it

7.4 Matching and match relations

Matching runs lexical first against preferred and approved labels after normalization, then embedding-assisted similarity with a confidence score. A customer concept links to the nearest core or domain concept by exactly one of four relations: **exact**, **close**, **broad** (core

concept is wider) or **narrow** (core concept is more specific). A synonym, abbreviation or misspelling is never a match relation; it is a label. Match relations are re-checked on every core release.

Four cases are flagged and never resolved automatically: the same label points to different concepts in one scope; a label equals another concept's preferred label; the same label means different things in two tenants (both stay scoped, and reviewers see the pair as evidence); a candidate has no close match.

7.5 Triage

If the term is	It goes to	Approved by
The same meaning as an existing concept (synonym, abbreviation, plural, locale variant, misspelling)	A label record in the narrowest lexicon in which it is true	Ops reviewer
A homonym in this scope	Stays scoped, conflict flagged, disambiguation note	Domain steward
A new concept used by this customer only	The customer extension vocabulary, with a match relation	Domain steward
A new concept of general value, or a label seen across several customers or in regulation	A core or domain change request through the normal lifecycle	Governance council
Noise, out of scope, or personal data	Rejected with a reason; evidence kept	Ops reviewer

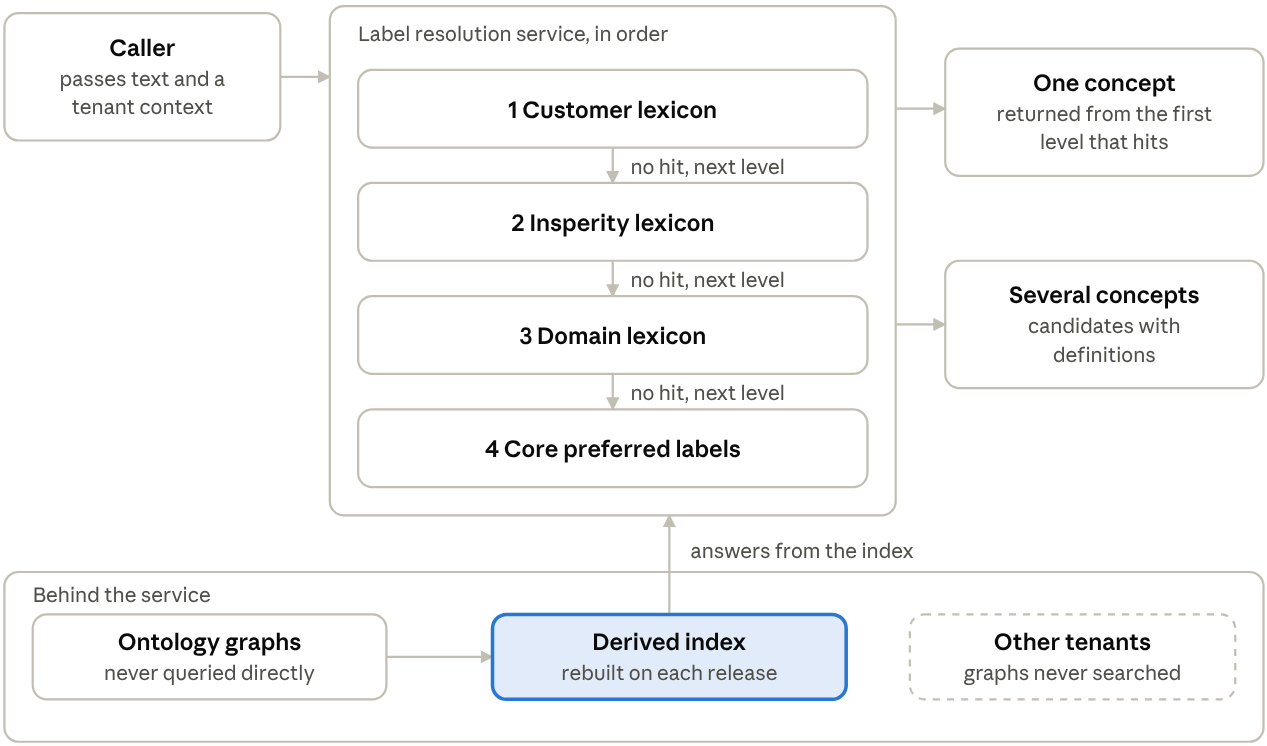
7.6 Label lifecycle, promotion and releases

A label starts Proposed, moves to In review, then Approved or Rejected. An approved label becomes Published only through a release; a published label can later be Deprecated. Every transition records who, when and why. A label that recurs across customers or appears in regulation can be promoted to a domain or core vocabulary as a universal alternate label by council decision and ships in a core release; a promoted label that later conflicts is demoted the same way. Lexicon changes release on their own cadence per scope; a lexicon release never changes the core version.

7.7 Resolution

Callers pass text and a tenant context. The label resolution service looks first in that customer's lexicon, then the Insperity lexicon, then the domain lexicon, then core preferred labels. It never searches another tenant's graphs, matches hidden labels without displaying them, and returns candidates with definitions when several concepts match at the same level. It answers from a derived index rebuilt on each release, not from the ontology graphs.

Resolution walks four levels in order and answers from a derived index



label resolution · 4 levels, 1 derived index

The service stops at the first level that hits. The dashed box is outside its reach by design.

8. Tenancy, personas and access

Each customer has its own named graphs, no repository is ever exposed, and what a person may do is decided from persona, job step, resource and context rather than a permission list.

8.1 Repositories and topology

Assuming GraphDB, one server per environment (dev, test, stage, prod) hosts a few repositories, split by reasoning and release cadence, not by tenant.

Repository	Holds	Reasoning	Why separate
`skl-core-domain`	Core, domain and operational-domain vocabularies, taxonomies, ontologies and mapping graphs, one named graph per asset	On	Released and reasoned together
`skl-lexicons`	Insperity lexicon and one named graph per tenant lexicon	Off or minimal	Labels stay out of reasoning and core releases
`skl-extensions`	Local concepts and match relations, one named graph per tenant	Light	Own release and deletion per tenant

The raw landing zone, candidate store and resolution index need not live in GraphDB. The core and domain repository is the same in every stage of growth; growth happens in the lexicon and extension repositories. Only where a customer contract or blast radius requires does a tenant move to a dedicated repository or server (`skl-tenant-` plus the tenant ID), and the move changes only its catalog record.

8.2 Isolation and privacy

1. **No direct access.** No repository is exposed to people, other systems or third parties. The store workbench and admin accounts are closed to normal use; break-glass access is time-bound and logged.
2. **Tenant tag from the first byte.** Every item carries a tenant ID from landing, and every query a tenant context.
3. **Isolation enforced by services.** Each tenant lexicon and extension vocabulary is its own named graph; services build the graph restriction into every query, accept no raw queries, and use separate read and write accounts. Store-level graph controls are a second layer.
4. **No cross-tenant path.** The one exception is promotion counting: reviewers see a count, never another tenant's evidence.

5. **Fail-closed screening.** Personal and sensitive content is detected, redacted or quarantined before extraction; a red-team set tests it before the pilot gate.
6. **Audit and exit.** Every read and write is logged. Offboarding removes the landing zone, graphs and index entries with no change to the core.

Written permission to use each customer's documents and feeds, a retention rule and data-use terms are conditions of onboarding.

8.3 Operational personas (proposed)

The persona and job catalog is owned by Personas & Jobs, which will supply fine-grained identity and access controls beyond traditional ACLs.

Persona	Jobs	Works on
Tenant provisioner	Create, suspend, remove a tenant; rotate service credentials	Tenant records and graphs, never content
Onboarding lead	Register sources, confirm data permission, run onboarding	One tenant's sources and ingest reports
Ops reviewer	Decide candidates, edit labels, raise escalations	Candidates and labels for assigned tenants
Domain steward	Approve match relations and local concepts; maintain vocabularies, taxonomies, mappings	Assets in the domains they steward
Ontologist	Propose changes to ontologies, triage rules, naming	Ontology assets, through the lifecycle
Platform operator	Run pipelines, releases, rollback, drift response	Services and releases, never content
Council member	Approve releases, label promotion, cross-domain conflicts	Release and promotion decisions
Auditor	Read the audit trail and decision records	Records only

8.4 How access is decided

Each request carries the person's persona assignments; the job and job step being performed; the resource (graph class, tenant, domain, item state); and the context (assignment, time). The service decides from rules authored in the SKL as governed rules, and logs the decision with the persona, job and step that authorized it. Rules a permission list cannot express:

- A reviewer decides only items assigned to them, in tenants they are provisioned for, while the item is In review.
- Nobody approves their own change; the proposing persona cannot be the approving one.
- The approval power of a steward ends at the domains they steward.
- A person with two personas has each persona's rights only while doing that persona's jobs.
- Elevated access is time-bound, logged and reviewed.

8.5 Provisioning

Provisioning is a persona job through the same screens and audit. **Tenant:** register; create lexicon and extension graphs, indexes and review queue; bind policies and persona assignments; confirm written data permission; activate. Deprovisioning reverses each step. **Person:** assign personas with tenant and domain scope and an end date; complete training and calibration; activate. Assignments are reassessed periodically and revoked at once for movers and leavers.

9. Semantic services and operations extensions

Five functions (capture, warehouse, map, manage, serve) are delivered by twelve semantic services; every graph is reached only through them, and screens, dashboards and later projects are built on their contracts.

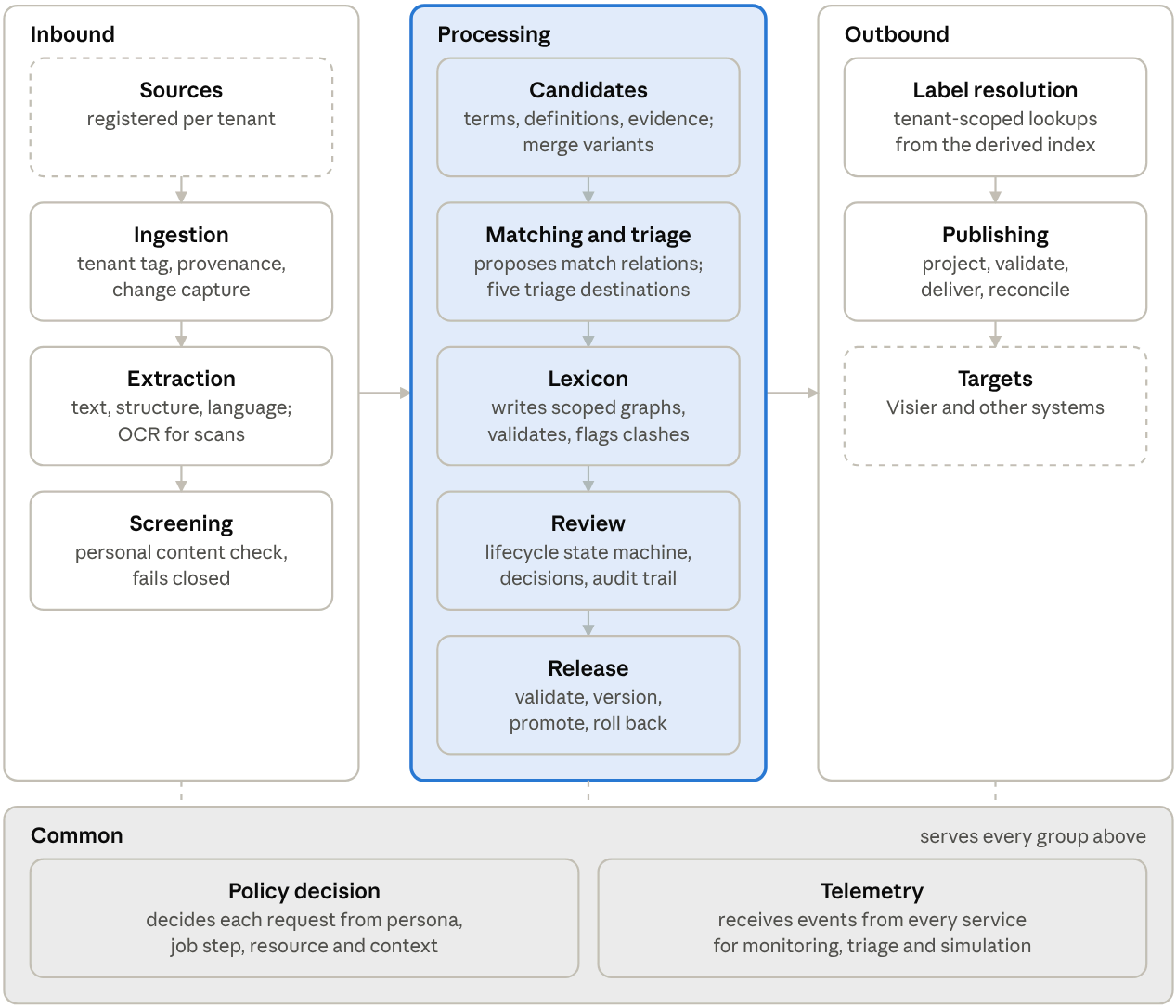
9.1 The services

Group	Service	Responsibility
Inbound	Ingestion	Lands each item with tenant tag, provenance and change capture, one connector per source type

Group	Service	Responsibility
Inbound	Extraction	Recovers text, structure and language; OCR for scans
Inbound	Screening	Detects personal and sensitive content before any other step; fails closed
Processing	Candidates	Extracts terms, defined terms and drafted definitions with evidence; normalizes and merges variants
Processing	Matching and triage	Matches candidates to concepts, proposes match relations, classifies each as label, local concept, core request or rejection
Processing	Lexicon	Writes label records and extension vocabularies to scoped graphs; validates shapes; detects conflicts
Processing	Review	Lifecycle state machine, assignments, decisions, audit trail
Processing	Release	Validation, versioned releases, promotion and rollback across environments; records changes as triples
Outbound	Label resolution	Tenant-scoped lookups from the derived index
Outbound	Publishing	Projects, validates, delivers and reconciles releases to targets through adapters
Common	Policy decision	Decides each request from persona, job step, resource and context
Common	Telemetry	Receives events from every service for monitoring, triage and simulation

Every service shares five rules: each request carries caller identity and tenant context; a policy decision is logged before any read or write; graph access is parameterized and no service accepts a raw query; each service emits events for request, decision, outcome and duration; the contract is versioned and changing it is a governed change.

Twelve services in four groups, with common services under all



semantic services · 12 services in 4 groups

Items move left to right: inbound services land and screen, processing services turn candidates into released vocabulary, and outbound services serve and publish it. The two common services sit under every group.

9.2 Capture and warehouse

Four source types are in scope: customer documentation (upload or file share), the content management system (API or export, incremental), customer data feeds (file, SFTP or API with declarative R2RML-style mappings) and employee handbooks (section-aware parsing, OCR). Each passes five steps: **land** (tenant tag, source, version, hash), **screen**

(fails closed; unclear items quarantined), **extract**, **propose** (a definition is drafted only when a source span supports it), **normalize**. Only the candidate record, with text, evidence span, source, tenant, frequency and confidence, leaves capture.

The warehouse keeps each store separate: raw landing (per tenant, never loaded into a graph), candidate store (working data, never released), core and domain graphs (versioned releases), Insperity lexicon, customer lexicons and customer extensions (own version per tenant), resolution index (disposable, rebuilt on release), and change records (append-only triples).

9.3 Publishing to targets

Publishing follows the SKL contract: publish, validate, reconcile, status and rollback, one adapter per target. For Visier, released vocabularies and taxonomies publish as dimension members and property values, each stamped with its SKL ID and version; synonyms and definitions stay in the SKL. Visier consumes and never edits; any difference on read-back is drift, corrected in the SKL.

Two points are not settled. The display-label rule (each member shows the tenant's approved label where one exists, otherwise the preferred label) is proposed for confirmation. Whether Visier can hold a tenant-specific name for a published member is unconfirmed; task W6-01 asks Visier, and if the answer is no the preferred label is published and customer terms resolve in Copilot, Compass and search. Visier limits come from public documentation and should be checked against the tenant.

9.4 Operations extensions

These are built on the service contracts, add no route to a graph, and can change without changing the baseline.

Extension	What it adds	Calls	Used by
Review and maintenance screens	Governed data entry for labels, local concepts, vocabularies, taxonomies, ontologies, mappings	Review, Lexicon, Release, Policy decision	Ops reviewers, stewards, ontologists

Extension	What it adds	Calls	Used by
Provisioning	Tenant and person lifecycles, persona assignment, recertification	Policy decision, Lexicon, Telemetry	Tenant provisioner, onboarding lead
Monitoring dashboards	Pipeline, processing quality, review work, service health, access and security, releases, capacity	Telemetry	Platform operators, ops review lead, auditors, council
Triage	One queue of alerts and exceptions with severity, owner and runbook	Telemetry, Review	Platform operators, ops review lead
Simulation	What-if runs showing the impact of a change before it is made	Release, Matching, Publishing, Telemetry	Stewards, ontologists, council, program manager

The review screens are the candidate queue, evidence viewer, concept picker, alternate-label editor, conflict panel, local-concept creator, lexicon browser, escalation and promotion inbox, and reviewer dashboard. Dashboards read telemetry, never the graphs. Operational triage (failed ingests, quarantines, drift, parity failures, repeated denials, expiring assignments, service-level breaches) is separate from candidate triage in Manage.

Simulation loads a proposed change into a draft graph and compares it with the current release without publishing. Workspaces use the reserved `sim` segment. The impact view shows concepts, labels, mappings and tenants affected, the objects that would change in each target, new conflicts, reviewer workload, and for capacity runs the predicted load, reasoning and lookup time. Candidate simulations: a core or domain release, label promotion or demotion, a changed match relation, an access rule, a new tenant or volume growth, and a Visier projection as a dry run.

10. Logic single source of truth

Business logic is defined once in a declarative spec under version control, compiled into what each runtime needs, and reached through one Decision API, so rules cannot drift between code, rules engines and ontologies.

The problem has three compounding forms: duplication (the same eligibility rule in a stored procedure, a Drools ruleset and a service-layer condition), silent drift (an incident fix applied in one place only) and no audit trail (rules edited directly in production with no review). Forcing everything into one tool fails, because procedural algorithms do not belong in a rules engine and stateful workflows do not belong in an ontology.

10.1 Right home by layer

Category	Question it answers	Home	Example
Constraints	What must always be true about the domain data?	Ontologies (OWL/RDF), schemas (JSON Schema, OpenAPI, Pydantic, Protobuf)	An order has at least one line item and the customer exists
Business rules	What actions or decisions follow from state?	Rules engines (Drools, Camunda, DMN) or a DSL	Gold tier and order over $100 gets 15% off
Computation	How is state computed or processed in sequence?	Code: microservices, DDD domain services	Route optimization, a multi-step feature pipeline

10.2 Governance pipeline

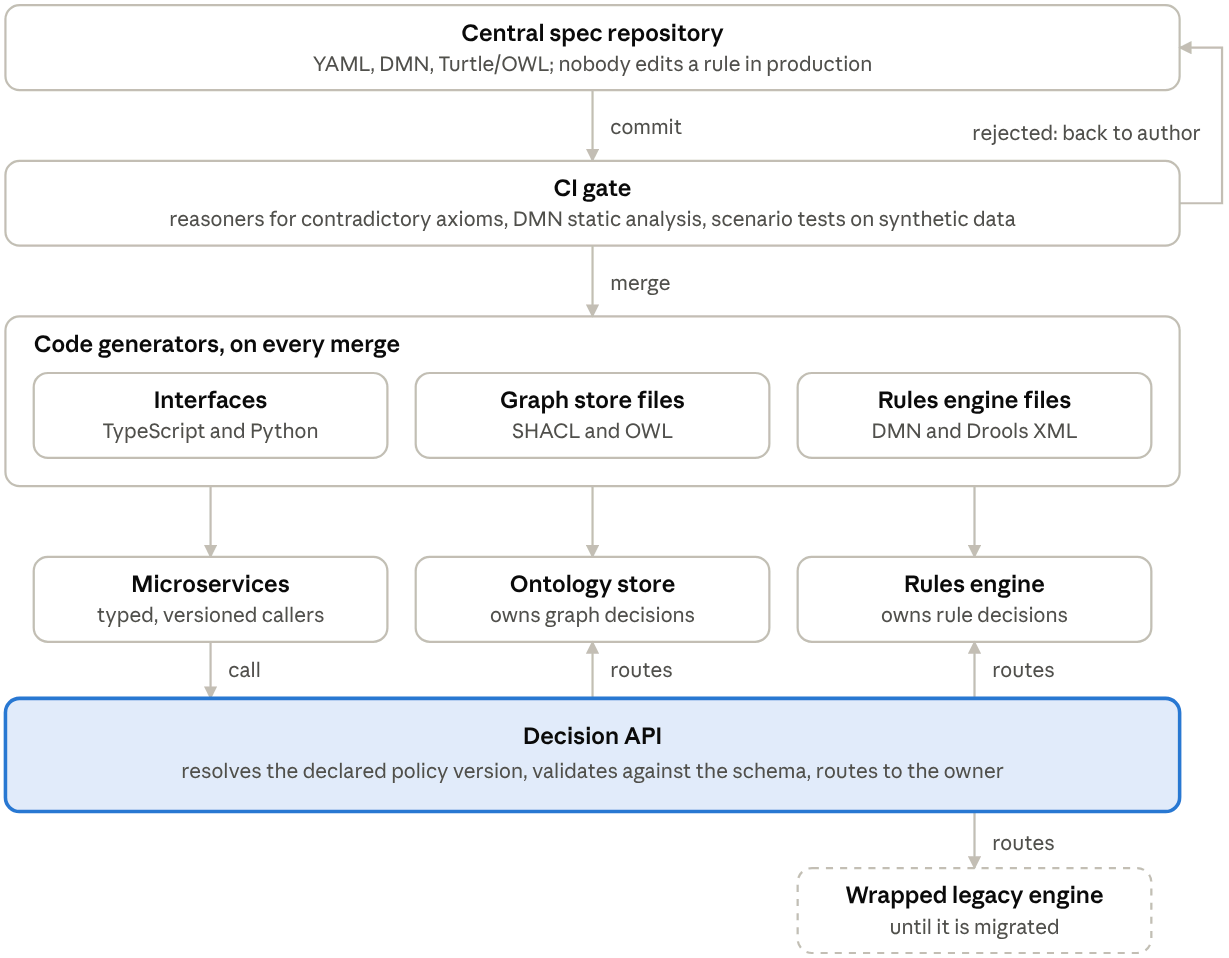
Specs live in a central repository in human-readable form (YAML, DMN, Turtle/OWL). Nobody edits a rule in a rules-engine screen or a production database; everything is committed first and reaches production only through CI. CI runs semantic reasoners (for example HermiT) for contradictory axioms and static analysis for malformed or conflicting DMN tables, then scenario tests against synthetic datasets. Rule sets and schemas carry semantic versions, and each consuming service declares the version it depends on, so a change cannot silently break a consumer. Rejected changes return to the author.

10.3 Generated executables and the Decision API

On every merge, code generators produce TypeScript and Python interfaces for microservices, SHACL/OWL files for the graph store, and DMN/Drools XML for the rules engine, all from the same spec. Drift becomes a build failure, not a production incident.

At run time callers do not reach the rules engine, ontology store or a legacy procedure directly. They call the Decision API, which resolves the caller's declared policy version, validates the request against the current schema, and routes to whichever backend owns the decision, including a wrapped legacy engine not yet migrated.

One spec, generated executables, one Decision API



logic single source of truth · spec to Decision API

Only the spec repository is edited by people. Everything below it is generated or routed, and the dashed box is the legacy engine that the strangler-fig migration retires.

10.4 Migration

1. **Discover and classify.** Inventory and audit with tracing and static analysis; log every point where a business decision is made; sort each into constraint, rule or computation.
2. **Decouple behind the API (strangler fig).** Wrap legacy engines and stored procedures behind the Decision API before touching internals; extract hardcoded logic into configuration or DMN piece by piece, re-pointing the facade as each piece lands.
3. **Schemas first, rules second.** Unify data definitions through a common ontology or schema registry, then consolidate rules; many apparent rule conflicts are data-shape conflicts.

Start with the repository and CI pipeline before extracting any rule; pilot on one bounded decision such as discount eligibility; run Phase 1 broadly before committing to Phase 2; make version declaration mandatory from day one.

10.5 Relation to compiled governance

The same write-once, compile-per-runtime pattern appears in section 11 for semantic governance rules. Treating the two as one pipeline is a proposed alignment, not a statement in either source document.

11. Runtime governance

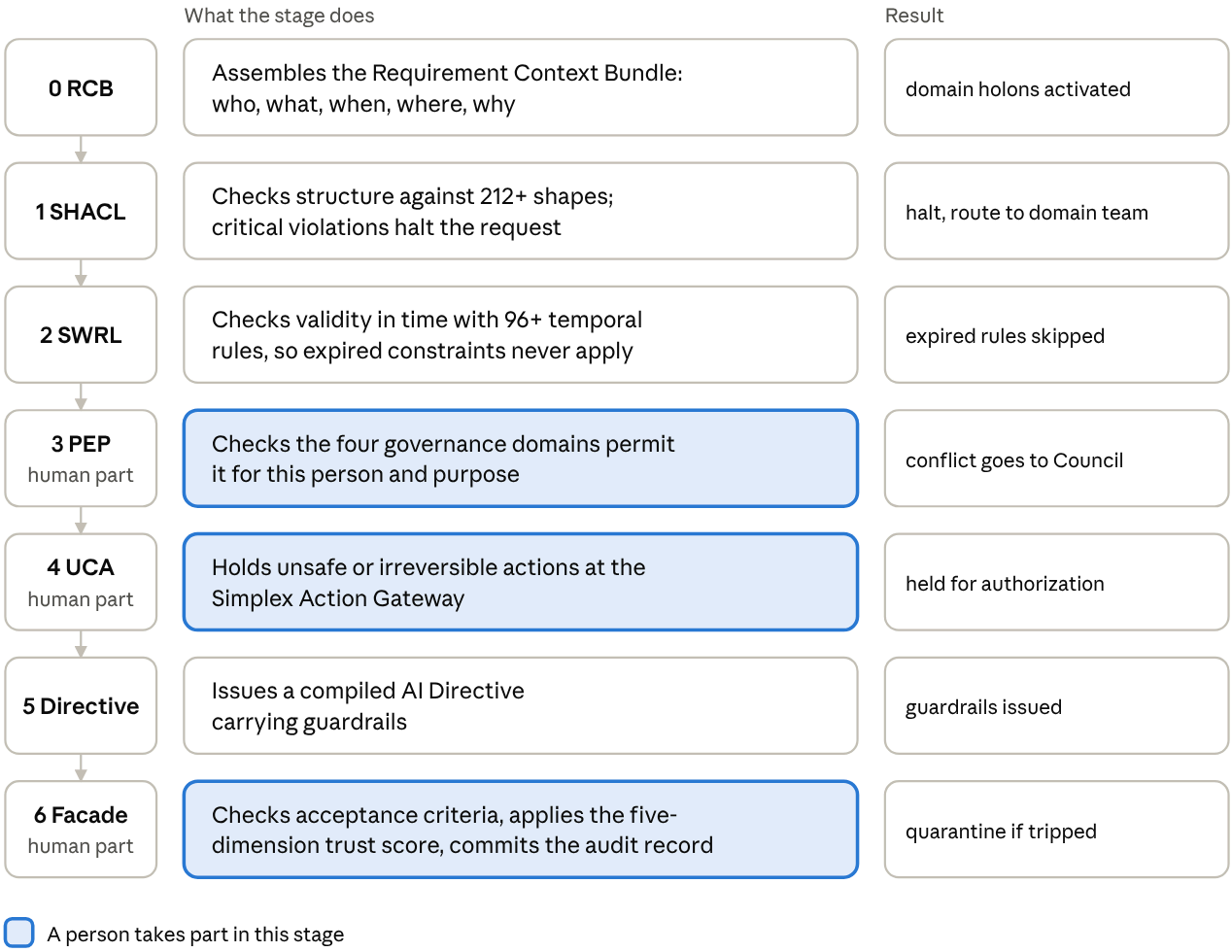
Every request and AI action takes the same seven-stage path through the gateway, rules are compiled once to where they are enforced, and agents do the continuous watching while people hold the decisions that carry judgment or risk.

11.1 Gateway stages (RDSG)

Stage	Name	What it does	Human part
0	RCB	Assembles the Requirement Context Bundle: who, what, when, where, why; activates domain holons	
1	SHACL	Checks structure; 212+ shapes; critical violations halt and route to the Domain Governance Team	
2	SWRL	Checks validity in time with 96+ temporal rules, so expired constraints are never applied	

Stage	Name	What it does	Human part
3	PEP	Checks the four governance domains permit it for this person and purpose; cross-jurisdiction conflicts go to the Enterprise Governance Council	Yes
4	UCA	Holds unsafe or irreversible actions at the Simplex Action Gateway for human authorization	Yes
5	Directive	Issues a compiled AI Directive carrying guardrails	
6	Facade	Checks acceptance criteria, applies the five-dimension trust score, commits the audit record to the Governance Named Graph, quarantines tripped outputs	Yes

Every request crosses seven stages, and three involve a person



gateway traversal · 7 stages, 3 with a human part

Stages run in order. A critical failure at stage 1 stops the request, and the three highlighted stages need a person.

11.2 Compiled governance

Rules are authored once as SHACL shapes, SWRL rules and LKIF-style deontic norms (obligation, permission, prohibition), harmonized by the Rule Harmonization Engine into one conflict-checked contract. The Rule Projection Agent compiles them to runtime targets (SQL CHECK constraints and dbt tests, OPA policies, vector-store filters, agent guardrails), versions each artifact with a KGCL record, and keeps a compiled policy registry and cache so the graph reasoner stays off the request path. Roles come from ontological definitions, not hand-edited permission lists.

11.3 Agents and loop

Eight agents operate: lineage ingestion, mapping confidence, rule projection, ontology alignment (HITL-gated), drift detection, RAG grounding, compliance monitoring (HITL-gated) and recursive learning (HITL-gated). The loop is Observe, Reason, Propose, HITL review, Adapt. Agents never write governed graphs directly.

11.4 Human checkpoints

Checkpoint	Decided by
Ontology change	Domain Architect with the Council
Weak mapping below threshold	Knowledge or Data Steward
Rule conflict across jurisdictions	Enterprise Governance Council
Irreversible action	Safety Reviewer
Low-trust output (quarantined)	Domain Governance Team
Candidate term (label or reject)	Ops reviewer
Match relation or local concept	Domain steward
Label promotion, core request, cross-domain conflict	Governance council

In the lexicon layer, extraction and matching propose and reviewers decide: the same rule that governs the eight agents. Persona names are illustrative; role definitions, not individuals, carry authority.

12. Measurement and assurance

Trust is measured, not assumed: MTBH gates AI actions, structural measures show whether the semantic design stays disciplined, and lexicon-layer targets prove the core stays small and tenants stay isolated.

12.1 MTBH

Mean Time Between Hallucinations is total operational time divided by verified hallucinations or breaker trips: MTBH = T_ops / N_h, as defined in the metrics registry of the AI Circuit Breaker v3.0 design. Track it under statistical process control; a falling MTBH is an early warning that the system is outside its validated envelope. The baseline uses 32 measures, a 150-question gold set baselined at Gate 1, and SPC monitoring on MTBH.

12.2 Structural measures

Dashboards read telemetry only, so each measure is emitted as a telemetry event by the check that computes it.

Measure	What it shows
Correspondence density	Lateral links per concept; too dense drifts toward a meshed graph, too sparse means fragmentation
Stem-reuse ratio	Share of new terms built from existing stems
Impact radius	Terms affected by a core change, before approval
Polyhierarchy rate	Concepts with more than one parent; reported above two
Domain-minted primitive share	Primitives minted outside core; should stay minimal
Promotion-backlog age	How long pending terms wait; long waits mean HITL capacity is the bottleneck

Measure	What it shows
HITL disposition mix	Label, domain concept, core change, reject; a shifting mix suggests gaps
Orphan rate per graph	Local concepts with no derivation path; target 0, checked in every tenant graph

12.3 Lexicon-layer targets

The claims are unproven until measured. Spike W2-02 builds a synthetic core and holds the same labels two ways, in the core and in scoped lexicons, measuring load, reasoning and query time, index size and release size at rising label and tenant volumes. Pass thresholds are agreed first (W1-04) so the result cannot move them; scale test W7-03 repeats it on the built system. Customer counts and labels per customer are not yet known, so the decision record states the synthetic parameters.

Measure	Target	Tested in
Customer-driven triples added to the core per onboarding	0	Lexicon builder; second-customer onboarding
Core load and reasoning time at top test volume	Within a tolerance set before the spike	W2-02, W7-03
P95 label lookup time	Threshold set from the spike	W2-02; Label Resolution Service
Cross-tenant label results	0	Isolation tests
Core releases triggered by a label change	None	Release pipeline
Label records with source, reviewer and release	100%	Review service

Also reported per customer and per release: lead time from first source to approved lexicon; candidates decided per reviewer-hour and age of the oldest open item; precision and recall of extraction, matching and triage on the gold set with reviewer agreement; drift incidents per release and parity failures caught before publish. Review service levels are set from the pilot, not guessed.

12.4 Lifecycle fit

HSGA governs the information baseline across the systems-engineering V-model, from stakeholder needs through operations, so requirements, architecture and design items are machine-checked early and monitored after fielding. Mapped from ISO/IEC 15288, the INCOSE SE Handbook and DoD AAF; artifact IDs are in v4.2.

13. Roadmap, decisions and risks

Delivery is phased and gated by measured evidence; three lexicon-layer decisions await council approval and the register below holds everything else open.

13.1 Programme phases

Phase	Focus	Exit evidence
1. Seed and baseline	Seed core from standards, 50+ shapes, charter the council, set trust baselines, stand up model registry	Gate 1 baseline: 150-question gold set; structural measures emitting telemetry
2. Extend and certify	Cross-domain extensions, 200+ shapes, external alignments, certification, ontology-driven access	Per-tenant graphs onboarded; 100% HITL compliance
3. Scale and self-serve	SHACL compliance above 99%, self-service extension toolkit, full deontic library, continuous trust monitoring	Anti-collapse protocol validated

13.2 Semantic services pilot and gates

The pilot uses one customer, one instance of each of the four source types, the downturn workforce advisory vocabularies from the SKL proposal, and Visier as first target. Planning dates assume a 9 November 2026 start and a pause from 21 December to 3 January.

Gate	Planning date	What is true when it passes
G0	21 Dec 2026	Label architecture decided on benchmark evidence; council seated; naming, ID and multi-parent rules agreed; inbound, processing and publishing contracts published; pilot customer, stewards and data permission confirmed
G1	8 Mar 2027	All four source types ingested with provenance on every record; no personal-data leaks in the red team; candidates triaged into scoped lexicons; zero customer-driven triples in the core; zero cross-tenant results
G2	3 May 2027	Pilot vocabularies live in Visier in one version with parity and reconcile checks clean; pilot customer lexicon reviewed by operations and published for its tenant
G3	24 May 2027	Scale test passed; second customer onboarded from the playbook with no change to the core; second target live; runbooks drilled

13.3 Decision register

Change the status with the dropdown and record the version in which it changed.

ID	Decision	Status	Note
D1	Primitives need BFO lineage, minimal, top-level domains only	Decided	Reflected in invariants
D2	Labels and tenant concepts live in separate graphs, never in core or domain ontologies	Decided	Refined by the lexicon layer
D3	MTBH is defined by the AI Circuit Breaker v3.0 metrics registry	Decided	
D4	Nested-domain primitive minting: do area containers count as top level?	Open	

ID	Decision	Status	Note
D5	Persona search ranking wording must match graph-scoped resolution	Open	
D6	Simulation sequencing against the roadmap, and which simulation is built first	Open	Label promotion, core release, Visier projection or capacity planning
D7	Council approver role, and who approves a customer term that maps to nothing in the core	Open	Listed open in one section, defined in the roles table
D8	Monitoring pattern: dashboards read telemetry only, but the guide reads model files	Open	
D9	Adopt the lexicon-layer pattern (scoped lexicons, extension vocabularies, derived index)	Proposed	Decision requested
D10	Adopt the five-function service model as baseline for every customer	Proposed	Decision requested
D11	Run benchmark spike W2-02 before build, thresholds set first in W1-04	Proposed	Decision requested
D12	Tenant ID format (t plus 8 hex) and name length limit (40 proposed)	Open	
D13	Persona-specific labels: stay on terms when true for every tenant, move to a lexicon otherwise?	Open	
D14	May an ontology exist at the enterprise tier?	Open	
D15	One catalog graph, or one per repository?	Open	
D16	URI path pattern for lexicon and mapping assets	Open	

13.4 Risks and mitigations

Risk	Mitigation
Lexicon layer adds a run-time lookup the core did not need	Derived index; P95 threshold set before the spike; spike and scale test
Extraction floods reviewers with low-value candidates	Confidence floors, de-duplication, gold set, triage precision target, bulk actions
Reviewers approve a label at the wrong scope	Narrowest scope default; conflict panel; steward escalation; promotion only through the council
Extension concepts drift from the core	Re-check match relations on every core release; drift report
Customer documents hold personal or confidential content	Written permission; fail-closed screening; retention rule; red-team test
Visier cannot hold tenant-specific names, or its Dimensions API (Alpha) is not ready	Ask early (W6-01); publish preferred label; file-based or Direct Data Intake fallback; labels stay in the SKL
Volumes unknown, so the spike is unrepresentative	Record synthetic parameters; repeat at scale test and final gate with real counts
Labels go stale	Effective dates, deprecation, periodic review of unused labels
Service-layer authorization flaw exposes one tenant to another	Parameterized queries only, separate read and write accounts, isolation tests in the pipeline, store graph controls as backstop
Store admin or workbench access bypasses the services	Private network, workbench closed, time-bound logged break-glass, encrypted backups
Persona and job rules too coarse or tangled to maintain	Author as governed rules with test cases; review with Personas & Jobs; pilot on reviewer and steward personas

13.5 Open questions

- Which customer is the pilot, and which CMS and feed formats does it use?

- Which graph store, review-application host and CI platform will the baseline use?
- How many customers and labels per customer should the spike and scale test assume?
- Who staffs operations review, and how many hours a week?
- Which languages beyond English appear in the pilot sources?
- What are the pass thresholds for core load, reasoning time and lookup time?
- Which identity provider and policy engine make access decisions, and where are persona and job assignments held?
- Who owns the persona and job catalog, and how are changes released?
- What is the break-glass procedure for store admin access, and who approves it?
- Which telemetry platform holds service events and audit records, and for how long?

14. Maintenance guide

Every change follows the same five steps, and its size decides the version number. This process is proposed; adjust it when the council sets its own.

14.1 Change classes

Class	Examples	Version effect
Major	Change an invariant; add or remove an asset type or tier; change a MUST rule; change the service contract model	New MAJOR, status resets to Draft
Minor	Add a service, persona, measure or section; settle an open decision without breaking an invariant	New MINOR
Revision	Wording, corrections, new examples	Note in the version history, no new number

14.2 Amendment steps

1. Describe the change and its class in a comment on the section it touches.
2. Edit the section. Keep numbering stable; add new subsections at the end of a section.

3. Add a row at the top of the version history: version, date, class, what changed, sources. Do not alter older rows.
4. Update the decision register: new decisions as Proposed or Open, settled ones as Decided with the version in the Note column.
5. Move the status in Document control to In review. Only the council sets Approved.

14.3 Keep in step

- The HSGA Explorer walkthrough, which mirrors sections 4 to 13.
- The graph naming standard and the Semantic Asset Metadata Standard, when a naming rule or asset type changes.
- The Semantic Services Baseline plan, when a gate date or service contract changes.

14.4 Conventions

- Example names, tenant IDs and customer terms are illustrative; label them so.
- A published asset name never changes; document a rename as a new asset and a retirement.
- Record facts from source documents with the source in the version history. Mark anything inferred as proposed.
- Use the document history to compare or restore earlier text instead of retyping it.

15. Sources and related documents

Version 5.0 draws only on these four documents; none is a web source.

Document	Date	Used for
Holonic Semantic Governance Architecture, v4.2, Reference Topology and Governance Diagram Edition	not recorded	Holonic theory, seven-layer holarchy, gateway, compiled governance, agents, measurement, lifecycle fit, standards foundation
Architecture Proposal: Alternate Labels and Customer Terminology	31 Oct 2026	Lexicon layer, services, tenancy, personas, access, operations extensions, pilot gates, risks

Document	Date	Used for
Graph Naming Standard: Insert for Semantic Governance Standards	1 Nov 2026	Asset types and tiers, names, graph classes, repository and tenant IDs, rules, amendments
Untangling Scattered Business Logic: an SSOT Architecture	not recorded	Logic classification, governance pipeline, generated executables, Decision API, migration

The lexicon proposal in turn cites the SKL proposal "SKL as Single Source of Truth, Publishing into Visier" (29 October 2026) and the PLM Lifecycle Dashboard for the SKL Semantic Services Baseline. Standards named: SKOS-XL, PROV-O, SHACL, R2RML (W3C), BFO (ISO/IEC 21838).

The "Key Pitfalls" table in the SSOT document is empty in the file provided; section 10 uses its problem statement instead.